

Generative Learning Algorithms

Sanjay Singh

School of Computer Engineering
Manipal Institute of Technology, MAHE
Manipal-576104, INDIA
sanjay.singh@manipal.edu

February 13, 2026

Sanjay Singh

Generative Learning Algorithms

Last module review

- Types of learning problems
 - Terminologies related to supervised learning
 - Linear regression
 - Important results from matrix calculus
 - Logistic regression
 - Fisher scoring
 - Generalized linear models
- Softmax regression

Sanjay Singh

Generative Learning Algorithms

- So far we have seen algorithms that model $p(y|x; \theta)$
- For example, logistic regression modeled $p(y|x; \theta)$ as $h_{\theta}(x) = g(\theta^T x)$

Classification Problem: Elephants vs Dogs

- Binary classification task
- Two classes:
 - Elephants: $y = 1$
 - Dogs: $y = 0$
- Goal: Distinguish between elephants and dogs based on features
- Given: Training set with labeled examples

Logistic Regression Approach

- Algorithm finds a **decision boundary**
- In simplest case: A **straight line** separating classes
- Boundary determined from training data
- Linear separation in feature space

Logistic regression finds optimal separating line

Making Predictions on New Data

- For new animal with features x :
 - 1 Check which side of decision boundary x falls on
 - 2 Predict elephant ($\hat{y} = 1$) if on “elephant side”
 - 3 Predict dog ($\hat{y} = 0$) if on “dog side”
- Simple decision rule based on boundary position

Training Phase:

Logistic regression learns boundary from labeled data

Prediction Phase:

New point → Check side of boundary → Prediction

Key Idea: Learn decision boundary from data, use for classification

Another approach

Building Separate Models for Each Class

Modeling Approach

- **For elephants:** Build model of “what elephants look like”
- **For dogs:** Build separate model of “what dogs look like”
- Each model learns characteristic features of its class

Elephant Model vs Dog Model

Training Process: Learning Class Characteristics

Elephants



Extract elephant features



Elephant Model

$P(\text{features} \mid \text{Elephant})$

Dogs



Extract dog features



Dog Model

$P(\text{features} \mid \text{Dog})$

Classification: Matching Against Both Models

For New Animal

- 1 Extract features from the new animal
- 2 Compare with **elephant model**: How elephant-like?
- 3 Compare with **dog model**: How dog-like?
- 4 Make decision based on best match

Decision Rule:

If new animal looks **more like elephants** than dogs $\Rightarrow$

Predict Elephant

If new animal looks **more like dogs** than elephants $\Rightarrow$

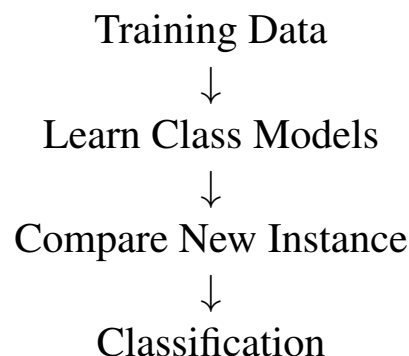
Predict Dog

Generative Learning Approach: Summary

Key Features:

- Models each class **separately**
- Creates “template” or “prototype” for each class
- Classification by **comparison** to both models
- Uses training examples to learn class characteristics

Flow:



Contrast with Previous Approach

Not finding a boundary but comparing to exemplars

- Algorithms that try to learn $p(y|x)$ are called **discriminative** learning algorithms
- Algorithms that try to model $p(x|y)$ (and $p(y)$) are called **generative** learning algorithms
- For instance y indicates whether an example is a dog (0) or an elephant (1)
- $p(x|y = 0)$ models the distribution of dog's features, and $p(x|y = 1)$ models the distribution of elephant's features

- Once $p(y)$ (called the **class priors**) and $p(x|y)$, GLA uses Bayes rule to derive the posterior distribution $p(y|x)$

$$p(y|x) = \frac{p(x|y)p(y)}{p(x)}$$

where $p(x) = p(x|y=0)p(y=0) + p(x|y=1)p(y=1)$

- If we are calculating $p(y|x)$ in order to make a prediction, then we don't need to calculate denominator, since

$$\begin{aligned} \arg \max_y p(y|x) &= \arg \max_y \frac{p(x|y)p(y)}{p(x)} \\ &= \arg \max_y p(x|y)p(y) \end{aligned}$$

Gaussian Discriminant Analysis (GDA)

We'll assume that $p(x|y)$ is distributed according to a multivariate normal distribution

- Multivariate normal distribution in n -dimension is parameterized by a **mean vector** $\mu \in \mathbb{R}^n$ and a **covariance matrix** $\Sigma \in \mathbb{R}^{n \times n}$ where $\Sigma \geq 0$ is symmetric and positive semi-definite
- Density of multivariate Gaussian distribution (also written as $\mathcal{N}(\mu, \Sigma)$) is given by

$$p(x; \mu, \Sigma) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right)$$

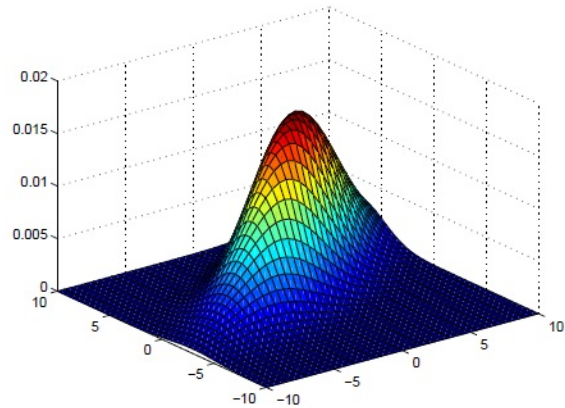
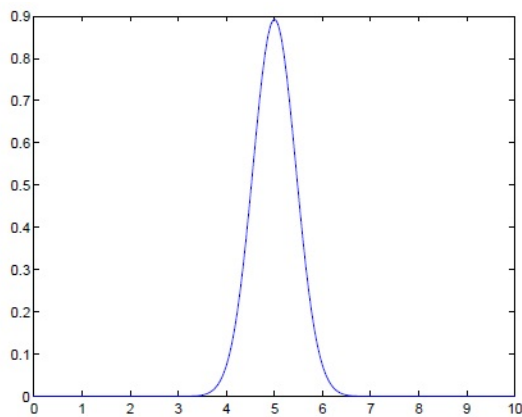


Figure 1: Gaussian density (a) Univariate (b) Multivariate with two variables

- For $X \sim \mathcal{N}(\mu, \Sigma)$, the mean is given by

$$E[X] = \int_{\mathbf{x}} \mathbf{x} p(\mathbf{x}; \mu, \Sigma) d\mathbf{x} = \mu$$

- Covariance for a pair of random variables X , and Y is defined as

$$\text{Cov}[X, Y] = E[(X - E[X])(Y - E[Y])] = E[XY] - E[X]E[Y]$$

- Also, the covariance of a vector-valued random variable Z is defined as

$$\text{Cov}(Z) = E[(Z - E[Z])(Z - E[Z])^T] = E[ZZ^T] - E[Z](E[Z])^T$$

- If $X \sim \mathcal{N}(\mu, \Sigma)$, then $\text{Cov}(X) = \Sigma$

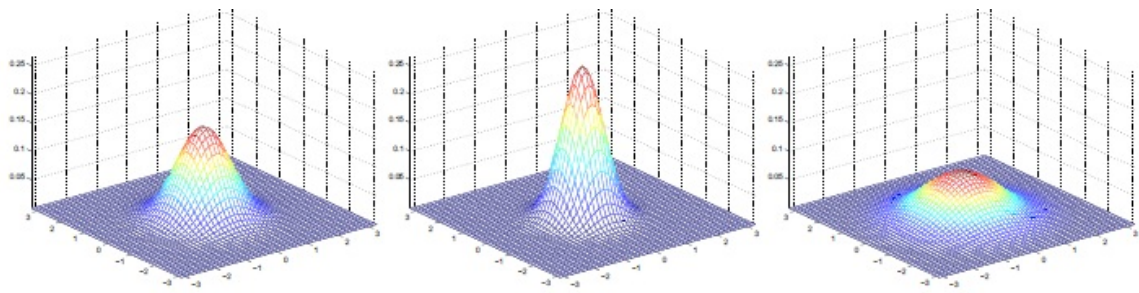


Figure 2: (a) Density with zero-mean and $\Sigma = I$ (called standard normal distribution) (b) Zero-mean and $\Sigma = 0.6I$ (c) zero-mean and $\Sigma = 2I$

As $\Sigma \uparrow$, the Gaussian becomes more “spread-out”, and as $\Sigma \downarrow$, the distribution becomes more “compressed”

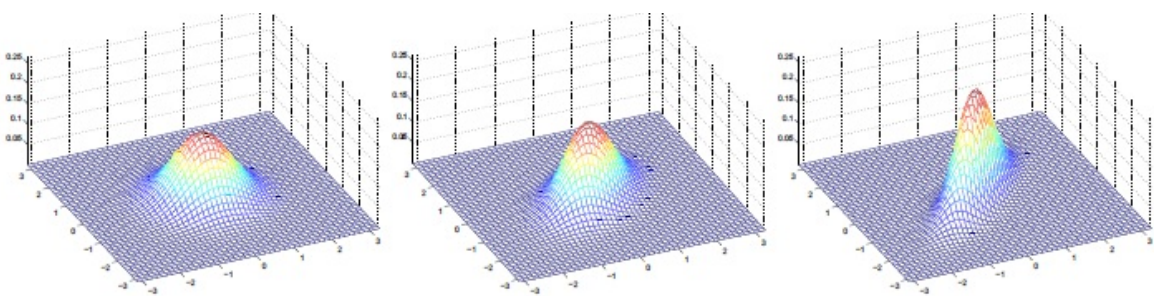


Figure 3: Gaussian density (a) $\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ (b) $\Sigma = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}$ (c) $\Sigma = \begin{bmatrix} 1 & 0.8 \\ 0.8 & 1 \end{bmatrix}$

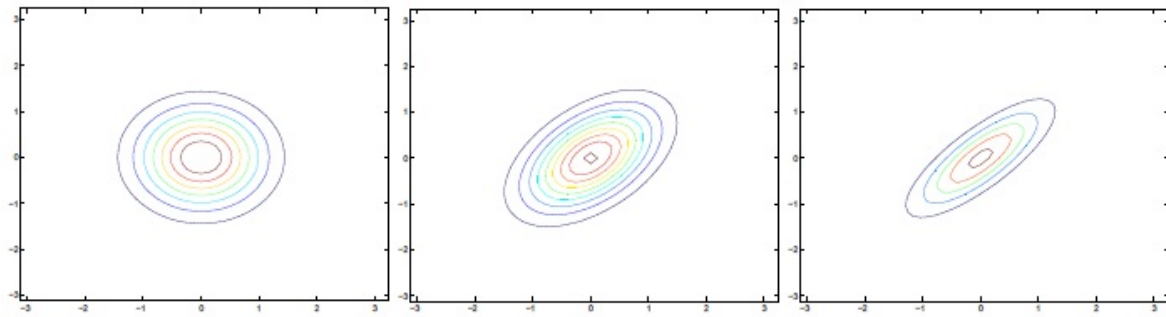


Figure 4: Contour plots-Gaussian density (a) $\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ (b) $\Sigma = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}$ (c)
 $\Sigma = \begin{bmatrix} 1 & 0.8 \\ 0.8 & 1 \end{bmatrix}$

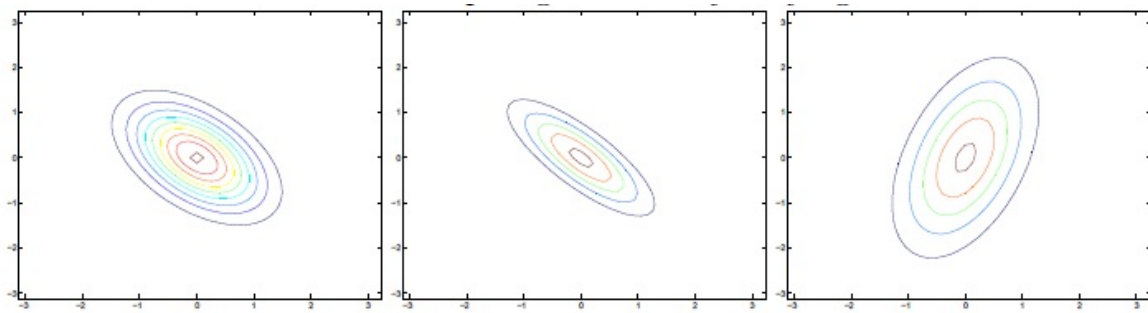


Figure 5: Gaussian density (a) $\Sigma = \begin{bmatrix} 1 & -0.5 \\ -0.5 & 1 \end{bmatrix}$ (b) $\Sigma = \begin{bmatrix} 1 & -0.8 \\ -0.8 & 1 \end{bmatrix}$ (c)
 $\Sigma = \begin{bmatrix} 3 & 0.8 \\ 0.8 & 3 \end{bmatrix}$

As we decrease the off-diagonal elements, the density becomes “compressed,” but in opposite direction

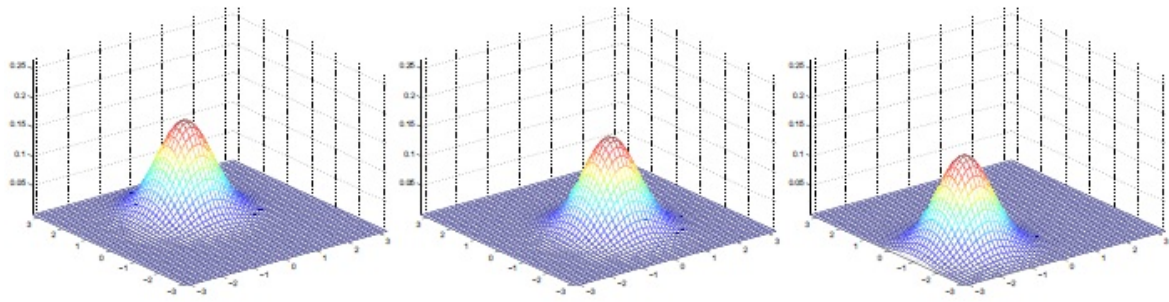


Figure 6: Gaussian density with $\Sigma = I$ and (a) $\mu = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ (b) $\mu = \begin{bmatrix} -0.5 \\ 0 \end{bmatrix}$ (c) $\mu = \begin{bmatrix} -1 \\ -1.5 \end{bmatrix}$

Diagonal Covariance Matrix Case

To get an intuition what a multivariate Gaussian is, consider a simple case where $n = 2$ and Σ is diagonal

- $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$, $\mu = \begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}$, and $\Sigma = \begin{bmatrix} \sigma_1^2 & 0 \\ 0 & \sigma_2^2 \end{bmatrix}$
- In this case, the multivariate Gaussian density has the form

$$\begin{aligned}
 p(x; \mu, \Sigma) &= \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right) \\
 &= \frac{1}{2\pi \begin{vmatrix} \sigma_1^2 & 0 \\ 0 & \sigma_2^2 \end{vmatrix}^{1/2}} \exp \left(-\frac{1}{2} \begin{bmatrix} x_1 - \mu_1 \\ x_2 - \mu_2 \end{bmatrix}^T \begin{bmatrix} \sigma_1^2 & 0 \\ 0 & \sigma_2^2 \end{bmatrix}^{-1} \begin{bmatrix} x_1 - \mu_1 \\ x_2 - \mu_2 \end{bmatrix} \right) \\
 &= \frac{1}{2\pi (\sigma_1^2 \cdot \sigma_2^2 - 0.0)^{1/2}} \exp \left(-\frac{1}{2} \begin{bmatrix} x_1 - \mu_1 \\ x_2 - \mu_2 \end{bmatrix}^T \begin{bmatrix} \frac{1}{\sigma_1^2} & 0 \\ 0 & \frac{1}{\sigma_2^2} \end{bmatrix} \begin{bmatrix} x_1 - \mu_1 \\ x_2 - \mu_2 \end{bmatrix} \right)
 \end{aligned}$$

$$\begin{aligned}
p(x; \mu, \Sigma) &= \frac{1}{2\pi(\sigma_1^2 \cdot \sigma_2^2 - 0.0)^{1/2}} \exp \left(-\frac{1}{2} \begin{bmatrix} x_1 - \mu_1 \\ x_2 - \mu_2 \end{bmatrix}^T \begin{bmatrix} \frac{1}{\sigma_1^2} & 0 \\ 0 & \frac{1}{\sigma_2^2} \end{bmatrix} \begin{bmatrix} x_1 - \mu_1 \\ x_2 - \mu_2 \end{bmatrix} \right) \\
&= \frac{1}{2\pi\sigma_1\sigma_2} \exp \left(-\frac{1}{2} \begin{bmatrix} x_1 - \mu_1 \\ x_2 - \mu_2 \end{bmatrix}^T \begin{bmatrix} \frac{1}{\sigma_1^2}(x_1 - \mu_1) \\ \frac{1}{\sigma_2^2}(x_2 - \mu_2) \end{bmatrix} \right) \\
&= \frac{1}{2\pi\sigma_1\sigma_2} \exp \left(-\frac{1}{2\sigma_1^2}(x_1 - \mu_1)^2 - \frac{1}{2\sigma_2^2}(x_2 - \mu_2)^2 \right) \\
&= \frac{1}{\sqrt{2\pi}\sigma_1} \exp \left(-\frac{1}{2\sigma_1^2}(x_1 - \mu_1)^2 \right) \cdot \frac{1}{\sqrt{2\pi}\sigma_2} \exp \left(-\frac{1}{2\sigma_2^2}(x_2 - \mu_2)^2 \right) \\
&= p(x_1; \mu_1, \sigma_1) \cdot p(x_2; \mu_2, \sigma_2)
\end{aligned}$$

In general, an n -dimensional Gaussian with mean $\mu \in \mathbb{R}^n$ and diagonal covariance matrix $\Sigma = \text{diag}(\sigma_1^2, \sigma_2^2, \dots, \sigma_n^2)$ is the same as a collection of n independent Gaussian random variables with mean μ_i and variance σ_i , respectively.

Gaussian Discriminant Analysis Model

- In a classification problem, when the input features x are continuous-valued random variables, we can use GDA model
- GDA models $p(x|y)$ using a multivariate normal distribution
- The model is

$$\begin{aligned}
y &\sim \text{Bernoulli}(\phi) \\
x|y = 0 &\sim \mathcal{N}(\mu_0, \Sigma) \\
x|y = 1 &\sim \mathcal{N}(\mu_1, \Sigma)
\end{aligned}$$

- Writing out the distributions

$$\begin{aligned}
p(y) &= \phi^y (1 - \phi)^{(1-y)} \\
p(x|y = 0) &= \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu_0)^T \Sigma^{-1} (x - \mu_0) \right) \\
p(x|y = 1) &= \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu_1)^T \Sigma^{-1} (x - \mu_1) \right)
\end{aligned}$$

- Model parameters: ϕ, μ_0, μ_1 and Σ

- The log-likelihood of the data is given by

$$\begin{aligned}
 l(\phi, \mu_0, \mu_1, \Sigma) &= \log \prod_{i=1}^m p(x^{(i)}, y^{(i)}; \phi, \mu_0, \mu_1, \Sigma) \\
 &= \log \prod_{i=1}^m p(x^{(i)} | y^{(i)}; \mu_0, \mu_1, \Sigma) p(y^{(i)}; \phi)
 \end{aligned}$$

- We need to find the maximum likelihood estimates of ϕ, μ_0, μ_1 and Σ

Expression for the log-likelihood of the training data

$$\begin{aligned}
 l(\phi, \mu_0, \mu_1, \Sigma) &= \log \prod_{i=1}^m p(x^{(i)} | y^{(i)}; \mu_0, \mu_1, \Sigma) p(y^{(i)}; \phi) \\
 &= \sum_{i=1}^m \log \left(p(x^{(i)} | y^{(i)}; \mu_0, \mu_1, \Sigma) p(y^{(i)}; \phi) \right) \\
 &= \sum_{i=1}^m \log p(x^{(i)} | y^{(i)}; \mu_0, \mu_1, \Sigma) + \sum_{i=1}^m \log p(y^{(i)}; \phi) \\
 &= \sum_{i=1}^m \log \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x^{(i)} - \mu_{y^{(i)}})^T \Sigma^{-1} (x^{(i)} - \mu_{y^{(i)}}) \right) \\
 &\quad + \sum_{i=1}^m \log \phi^{y^{(i)}} (1 - \phi)^{1-y^{(i)}} \\
 &= \sum_{i=1}^m \log \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} + \log \exp \left(-\frac{1}{2} (x^{(i)} - \mu_{y^{(i)}})^T \Sigma^{-1} (x^{(i)} - \mu_{y^{(i)}}) \right) \\
 &\quad + \sum_{i=1}^m \left(y^{(i)} \log \phi + (1 - y^{(i)}) \log(1 - \phi) \right)
 \end{aligned}$$

$$\begin{aligned}
l(\phi, \mu_0, \mu_1, \Sigma) &= \sum_{i=1}^m \log \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} + \log \exp \left(-\frac{1}{2} (x^{(i)} - \mu_{y^{(i)}})^T \Sigma^{-1} (x^{(i)} - \mu_{y^{(i)}}) \right) \\
&\quad + \sum_{i=1}^m \left(y^{(i)} \log \phi + (1 - y^{(i)}) \log(1 - \phi) \right) \\
&= \sum_{i=1}^m \left(\frac{1}{2} \log \frac{1}{|\Sigma|} - \frac{1}{2} (x^{(i)} - \mu_{y^{(i)}})^T \Sigma^{-1} (x^{(i)} - \mu_{y^{(i)}}) \right) \\
&\quad + \sum_{i=1}^m \left(y^{(i)} \log \phi + (1 - y^{(i)}) \log(1 - \phi) \right)
\end{aligned}$$

- Constant terms independent of the parameter is ignored
- Now the likelihood is maximized by setting derivative of $l(\phi, \mu_0, \mu_1, \Sigma)$ wrt each of the parameter to zero

Maximum Likelihood Estimate of ϕ

$$\begin{aligned}
l(\phi, \mu_0, \mu_1, \Sigma) &= \sum_{i=1}^m \left(\frac{1}{2} \log \frac{1}{|\Sigma|} - \frac{1}{2} (x^{(i)} - \mu_{y^{(i)}})^T \Sigma^{-1} (x^{(i)} - \mu_{y^{(i)}}) \right) \\
&\quad + \sum_{i=1}^m \left(y^{(i)} \log \phi + (1 - y^{(i)}) \log(1 - \phi) \right) \\
\frac{\partial l}{\partial \phi} &= \sum_{i=1}^m y^{(i)} \cdot \frac{1}{\phi} + (1 - y^{(i)}) \cdot \frac{1}{(1 - \phi)} \times -1 \\
&= \sum_{i=1}^m \frac{y^{(i)}}{\phi} - \frac{1 - y^{(i)}}{1 - \phi} \\
&= \frac{\sum_{i=1}^m 1\{y^{(i)} = 1\}}{\phi} - \frac{m - \sum_{i=1}^m 1\{y^{(i)} = 1\}}{1 - \phi}
\end{aligned}$$

Now equating $\frac{\partial l}{\partial \phi}$ to zero, we get

$$\begin{aligned} \frac{\sum_{i=1}^m 1\{y^{(i)} = 1\}}{\phi} - \frac{m - \sum_{i=1}^m 1\{y^{(i)} = 1\}}{1 - \phi} &= 0 \\ \frac{\sum_{i=1}^m 1\{y^{(i)} = 1\}}{\phi} &= \frac{m - \sum_{i=1}^m 1\{y^{(i)} = 1\}}{1 - \phi} \\ (1 - \phi) \sum_{i=1}^m 1\{y^{(i)} = 1\} &= \phi \left(m - \sum_{i=1}^m 1\{y^{(i)} = 1\} \right) \\ \sum_{i=1}^m 1\{y^{(i)} = 1\} - \phi \sum_{i=1}^m 1\{y^{(i)} = 1\} &= m\phi - \phi \sum_{i=1}^m 1\{y^{(i)} = 1\} \\ \boxed{\phi = \frac{1}{m} \sum_{i=1}^m 1\{y^{(i)} = 1\}} \end{aligned}$$

Maximum Likelihood Estimation of μ_0 and μ_1

$$\begin{aligned} l(\phi, \mu_0, \mu_1, \Sigma) &= \sum_{i=1}^m \left(\frac{1}{2} \log \frac{1}{|\Sigma|} - \frac{1}{2} (x^{(i)} - \mu_{y^{(i)}})^T \Sigma^{-1} (x^{(i)} - \mu_{y^{(i)}}) \right) \\ &\quad + \sum_{i=1}^m \left(y^{(i)} \log \phi + (1 - y^{(i)}) \log(1 - \phi) \right) \\ \nabla_{\mu_0} l(\phi, \mu_0, \mu_1, \Sigma) &= \nabla_{\mu_0} \left(\sum_{i: y^{(i)}=0}^m \left(-\frac{1}{2} (x^{(i)} - \mu_0)^T \Sigma^{-1} (x^{(i)} - \mu_0) \right) \right) \\ &= -\frac{1}{2} \sum_{i: y^{(i)}=0}^m \nabla_{\mu_0} (x^{(i)} - \mu_0)^T \Sigma^{-1} (x^{(i)} - \mu_0) \\ &= -\frac{1}{2} \sum_{i: y^{(i)}=0}^m \nabla_{\mu_0} (x^{(i)} - \mu_0)^T (\Sigma^{-1} x^{(i)} - \Sigma^{-1} \mu_0) \\ &= -\frac{1}{2} \sum_{i: y^{(i)}=0}^m \nabla_{\mu_0} \left(x^{(i)T} \Sigma^{-1} x^{(i)} - x^{(i)T} \Sigma^{-1} \mu_0 - \mu_0^T \Sigma^{-1} x^{(i)} + \mu_0^T \Sigma^{-1} \mu_0 \right) \\ &= -\frac{1}{2} \sum_{i: y^{(i)}=0}^m \nabla_{\mu_0} \left(-x^{(i)T} \Sigma^{-1} \mu_0 - \mu_0^T \Sigma^{-1} x^{(i)} + \mu_0^T \Sigma^{-1} \mu_0 \right) \end{aligned}$$

$$\begin{aligned}
\nabla_{\mu_0} l(\phi, \mu_0, \mu_1, \Sigma) &= -\frac{1}{2} \sum_{i:y^{(i)}=0}^m \nabla_{\mu_0} \left(-x^{(i)T} \Sigma^{-1} \mu_0 - \mu_0^T \Sigma^{-1} x^{(i)} + \mu_0^T \Sigma^{-1} \mu_0 \right) \\
&= -\frac{1}{2} \sum_{i:y^{(i)}=0}^m \nabla_{\mu_0} \left(\mu_0^T \Sigma^{-1} \mu_0 - x^{(i)T} \Sigma^{-1} \mu_0 - x^{(i)T} \Sigma^{-1} \mu_0 \right) \\
&= -\frac{1}{2} \sum_{i:y^{(i)}=0}^m \nabla_{\mu_0} \left(\mu_0^T \Sigma^{-1} \mu_0 - 2x^{(i)T} \Sigma^{-1} \mu_0 \right) \\
&= -\frac{1}{2} \sum_{i:y^{(i)}=0}^m \left(2\Sigma^{-1} \mu_0 - 2(\Sigma^{-1})^T x^{(i)} \right) \\
&= -\frac{1}{2} \sum_{i:y^{(i)}=0}^m \left(2\Sigma^{-1} \mu_0 - 2\Sigma^{-1} x^{(i)} \right) \quad \text{since } \Sigma \in \mathbb{S}_{++}^n \\
&= - \sum_{i:y^{(i)}=0}^m \left(\Sigma^{-1} \mu_0 - \Sigma^{-1} x^{(i)} \right) \\
&= - \sum_{i=1}^m 1\{y^{(i)} = 0\} \Sigma^{-1} \mu_0 + \sum_{i=1}^m 1\{y^{(i)} = 0\} \Sigma^{-1} x^{(i)}
\end{aligned}$$

- Now, $\nabla_{\mu_0} l(\phi, \mu_0, \mu_1, \Sigma) = 0$, we get

$$\begin{aligned}
\sum_{i=1}^m 1\{y^{(i)} = 0\} \Sigma^{-1} \mu_0 &= \sum_{i=1}^m 1\{y^{(i)} = 0\} \Sigma^{-1} x^{(i)} \\
\Sigma^{-1} \mu_0 \sum_{i=1}^m 1\{y^{(i)} = 0\} &= \Sigma^{-1} \sum_{i=1}^m 1\{y^{(i)} = 0\} x^{(i)} \\
\cancel{\Sigma^{-1}} \mu_0 \sum_{i=1}^m 1\{y^{(i)} = 0\} &= \cancel{\Sigma^{-1}} \sum_{i=1}^m 1\{y^{(i)} = 0\} x^{(i)}
\end{aligned}$$

$$\boxed{\mu_0 = \frac{\sum_{i=1}^m 1\{y^{(i)} = 0\} x^{(i)}}{\sum_{i=1}^m 1\{y^{(i)} = 0\}}}$$

- Similarly, we can derive

$$\boxed{\mu_1 = \frac{\sum_{i=1}^m 1\{y^{(i)} = 1\} x^{(i)}}{\sum_{i=1}^m 1\{y^{(i)} = 1\}}}$$

- To simplify derivation, let $S = \Sigma^{-1}$
- Note that $|S| = \frac{1}{|\Sigma|}$
- The maximum likelihood estimate S_m would correspond to the actual maximum likelihood estimate Σ_m as

$$S_m^{-1} = \Sigma_m$$

$$l(\phi, \mu_0, \mu_1, \Sigma) = \sum_{i=1}^m \left(\frac{1}{2} \log \frac{1}{|\Sigma|} - \frac{1}{2} (x^{(i)} - \mu_{y^{(i)}})^T \Sigma^{-1} (x^{(i)} - \mu_{y^{(i)}}) \right) \\ + \sum_{i=1}^m \left(y^{(i)} \log \phi + (1 - y^{(i)}) \log(1 - \phi) \right)$$

Above equation can be re-written in terms of S , with non-relevant terms ignored as

$$\nabla_S l = \sum_{i=1}^m \nabla_S \left(\frac{1}{2} \log |S| - \frac{1}{2} \underbrace{(x^{(i)} - \mu_{y^{(i)}})^T}_{b_i^T} \Sigma^{-1} \underbrace{(x^{(i)} - \mu_{y^{(i)}})}_{b_i} \right) \\ = \sum_{i=1}^m \nabla_S \left(\frac{1}{2} \log |S| - \frac{1}{2} b_i^T S b_i \right) \\ = \sum_{i=1}^m \left(\frac{1}{2} \nabla_S \log |S| - \frac{1}{2} \nabla_S b_i^T S b_i \right)$$

$$\nabla_S l = \sum_{i=1}^m \left(\frac{1}{2} \nabla_S \log |S| - \frac{1}{2} \nabla_S b_i^T S b_i \right)$$

Using following matrix calculus identities

- $\nabla_A |A| = |A| (A^{-1})^T$
- $\nabla_A \log |A| = A^{-1}$
- $\nabla_A a^T A b = a b^T$

we get

$$\begin{aligned} \nabla_S l &= \sum_{i=1}^m \left(\frac{1}{2|S|} \nabla_S |S| - \frac{1}{2} \nabla_S b_i^T S b_i \right) \\ &= \sum_{i=1}^m \left(\frac{1}{2|S|} |S| (S^{-1})^T - \frac{1}{2} b_i b_i^T \right) \\ &= \sum_{i=1}^m \left(\frac{1}{2} (S^{-1})^T - \frac{1}{2} b_i b_i^T \right) \\ &= \sum_{i=1}^m \left(\frac{1}{2} S^{-1} - \frac{1}{2} b_i b_i^T \right) \quad \text{since } \Sigma \in \mathbb{S}_{++}^n \text{ so } (S^{-1})^T = S^{-1} \\ &= \frac{1}{2} \sum_{i=1}^m (\Sigma - b_i b_i^T) \end{aligned}$$

Now, $\nabla_S l = 0$, yields

$$\begin{aligned} \frac{1}{2} \sum_{i=1}^m (\Sigma - b_i b_i^T) &= 0 \\ \frac{1}{2} m \Sigma - \frac{1}{2} \sum_{i=1}^m b_i b_i^T &= 0 \\ \frac{1}{2} m \Sigma &= \frac{1}{2} \sum_{i=1}^m b_i b_i^T \\ \Sigma &= \frac{1}{m} \sum_{i=1}^m b_i b_i^T \end{aligned}$$

$$\Sigma = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu_{y^{(i)}})(x^{(i)} - \mu_{y^{(i)}})^T$$

By maximizing $l(\phi, \mu_0, \mu_1, \Sigma)$ wrt the parameters, we find the maximum likelihood estimates of parameters to be:

$$\phi = \frac{1}{m} \sum_{i=1}^m 1\{y^{(i)} = 1\}$$

$$\mu_0 = \frac{\sum_{i=1}^m 1\{y^{(i)} = 0\}x^{(i)}}{\sum_{i=1}^m 1\{y^{(i)} = 0\}}$$

$$\mu_1 = \frac{\sum_{i=1}^m 1\{y^{(i)} = 1\}x^{(i)}}{\sum_{i=1}^m 1\{y^{(i)} = 1\}}$$

$$\Sigma = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu_{y^{(i)}})(x^{(i)} - \mu_{y^{(i)}})^T$$

Implementation Code¹

Reading training data

```
using DataFrames;  
dtf1 = readtable("iris.csv")  
X = convert(Array, dtf1[:, [:x5_1, :x3_5, :x1_4, :x0_2]])  
Y = convert(Array, dtf1[:, [:x0]])
```

Computing ϕ

```
function calculate_phi(Y)  
    m = length(Y)  
    return count(x->x==1, Y) / m  
end
```

¹From Parag Radke post, <https://tinyurl.com/y4ed7waw>

Computing μ_0 , and μ_1

```
function calculate_mu0(X, Y)
m = length(Y)
y_neg = count(x->x==0, Y)
conditional_sum_x = zeros(X[1,:])
for i=1:m
    xi = X[i,:]
    yi = Y[i]
    conditional_sum_x = conditional_sum_x + (yi ==0?xi:zeros(xi))
end
return (1/m)*conditional_sum_x/y_neg
end

function calculate_mu1(X, Y)
m = length(Y)
y_pos = count(x->x==1, Y)
conditional_sum_x = zeros(X[1,:])
for i=1:m
    xi = X[i,:]
    yi = Y[i]
    conditional_sum_x =conditional_sum_x + (yi ==1?xi:zeros(xi))
end
return (1/m)*conditional_sum_x/y_pos
end
```

Computing Σ

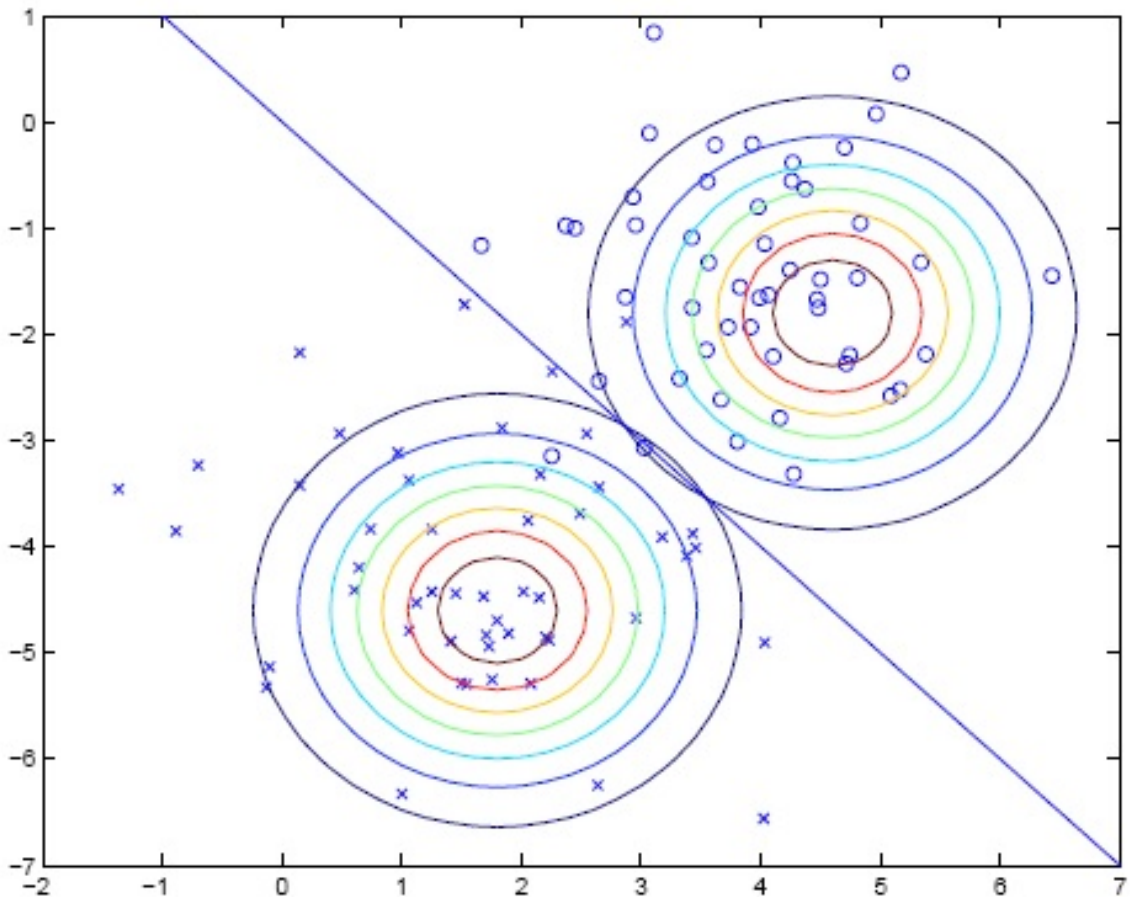
```
function calculate_sigma(X, Y, mu0, mu1)
m = length(Y)
sqr_sum = zeros(size(X)[2], size(X)[2])
for i=1:m
    xi = X[i,:]
    yi = Y[i]
    sqr = yi == 1? (xi-mu1)*(xi-mu1)':(xi-mu0)*(xi-mu0)'
    sqr_sum = sqr_sum + sqr
end
return sqr_sum/m
end
```

Computing multivariate distribution $p(x; \mu, \Sigma)$

```
function calculate_px_py(x, mu, sigma)
n = 1
pi = 3.14
dim = size(mu,1)
return ((1/(2*pi)^(dim/2)*sqrt(det(sigma)))*exp(-0.5*(x-mu)'*inv(sigma)*(x-mu)))[1]
end
```

Computing prior distribution, $p(y)$

```
function calculate_py(y, phi)
return y==1? phi : (1-phi)
end
```



Sanjay Singh

Generative Learning Algorithms

GDA and Logistic Regression

- We can view the quantity $p(y = 1|x; \phi, \mu_0, \mu_1, \Sigma)$ as function of x , we can express it in the form of

$$p(y = 1|x; \phi, \mu_0, \mu_1, \Sigma) = \frac{1}{1 + \exp(-\theta^T x)}$$

where θ is some appropriate function of ϕ, μ_0, μ_1 , and Σ
it is exactly same as the logistic regression-a discriminative algorithm used to model $p(y|x)$

- GDA and logistic regression will give different decision boundaries when trained on the same dataset
- Which one is better: GDA or logistic regression?

Sanjay Singh

Generative Learning Algorithms

- If $p(x|y)$ is multivariate Gaussian then $p(y|x)$ follows as logistic function, but the converse is not true i.e if $p(y|x)$ being a logistic does not imply $p(x|y)$ is multivariate Gaussian
- GDA makes stronger modelling assumptions about the data than logistic regression
- When $p(x|y)$ is Gaussian (with shared Σ), then GDA is **asymptotically efficient**
which means, for large training sets, there is no algorithm that is better than GDA (in terms of accurate estimation of $p(y|x)$)

- GDA makes stronger modeling assumptions and is more data efficient (i.e requires less training data to learn “well”) when modeling assumptions are correct or approximately correct
- Logistic regression makes weaker assumption and is more robust to deviation from modeling assumptions
- When dataset is large and non-Gaussian, logistic regression will always performs better than GDA

Example

RadioMirchi partitions radio station listeners into two groups-the 'young' and 'old'. They assume that, given the knowledge that a customer is either 'young' or 'old', is sufficient to determine whether or not a customer will like a particular radio station, independent of their likes or dislikes for any other stations.

Given that a customer is young, she has 95% chance to like Radio1, a 5% chance to like Radio2, a 2% chance to like Radio3 and a 20% chance to like Radio4. Similarly, an old listener has a 3% chance to like Radio1, an 82% chance to like Radio2, a 34% chance to like Radio3 and a 92% chance to like Radio4. It is known that 90% of the listeners are old.

Given this model, and the fact that a new customer likes Radio1 and Radio3, but dislikes Radio2 and Radio4, what is the probability that the new customer is young?

Given that for young listeners

$$p(r_1 = l | age = y) = 0.95; p(r_1 = d | age = y) = 0.05$$

$$p(r_2 = l | age = y) = 0.05; p(r_2 = d | age = y) = 0.95$$

$$p(r_3 = l | age = y) = 0.02; p(r_3 = d | age = y) = 0.98$$

$$p(r_4 = l | age = y) = 0.2; p(r_4 = d | age = y) = 0.8$$

$$p(y) = 0.1$$

Old listeners:

$$p(r_1 = l | age = o) = 0.03; p(r_1 = d | age = o) = 0.97$$

$$p(r_2 = l | age = o) = 0.82; p(r_2 = d | age = o) = 0.18$$

$$p(r_3 = l | age = o) = 0.34; p(r_3 = d | age = o) = 0.66$$

$$p(r_4 = l | age = o) = 0.92; p(r_4 = d | age = o) = 0.08$$

$$p(o) = 0.9$$

As per the query

$$p(age = y | r_1 = l, r_2 = d, r_3 = l, r_4 = d)?$$

From Bayes theorem

$$\begin{aligned} & p(\text{age} = y | r_1 = l, r_2 = d, r_3 = l, r_4 = d) \\ &= \frac{p(r_1 = l, r_2 = d, r_3 = l, r_4 = d | \text{age} = y)p(\text{age} = y)}{\sum_{\text{age}} p(r_1 = l, r_2 = d, r_3 = l, r_4 = d)} \\ &= \frac{p(r_1 = l | \text{age} = y)p(r_2 = d | \text{age} = y)p(r_3 = l | \text{age} = y)p(r_4 = d | \text{age} = y)p(\text{age} = y)}{p(r_1 = l, r_2 = d, r_3 = l, r_4 = d | \text{age} = y)p(\text{age} = y) + p(r_1 = l, r_2 = d, r_3 = l, r_4 = d | \text{age} = o)} \end{aligned}$$

Numerator part:

$$\begin{aligned} & p(r_1 = l | \text{age} = y)p(r_2 = d | \text{age} = y)p(r_3 = l | \text{age} = y)p(r_4 = d | \text{age} = y)p(\text{age} = y) \\ &= 0.95 \times 0.95 \times 0.2 \times 0.8 \times 0.1 \\ &= 0.01444 \end{aligned}$$

Denominator part

$$\begin{aligned} & p(r_1 = l, r_2 = d, r_3 = l, r_4 = d | \text{age} = y)p(\text{age} = y) \\ &+ p(r_1 = l, r_2 = d, r_3 = l, r_4 = d | \text{age} = o)p(\text{age} = o) = 0.01444 \quad (1) \\ &+ 1.3219 \times 10^{-4} \end{aligned}$$

which gives

$$p(\text{age} = y | r_1 = l, r_2 = d, r_3 = l, r_4 = d) = 0.9161 \quad (2)$$

Naive Bayes

- Consider feature vectors x_i to be discrete valued
- Consider building an email spam filter using machine learning
- Email classification problem is an example of **text classification**
- Lets say we have a training set
- We represent an email via a feature vector whose length is equal to the number of words in dictionary
- If an email contains the i -th word of the dictionary, then $x_i = 1$ otherwise $x_i = 0$

For example,

$$x = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix} \begin{matrix} a \\ aardvark \\ aardwolf \\ \vdots \\ buy \\ \vdots \\ zygmurgy \end{matrix}$$

- Represents an email that contain the words “a”, and “buy” but not “aardvark,” “aardwolf” or “zygmurgy”
- Set of words encoded into the feature vector is called the **vocabulary**
- So $x \in \{0, 1\}^{VocabularySize}$

- Having chosen feature vector, we want to build a generative model, $p(x|y)$
- Suppose there is a vocabulary of 50000 words then $x \in \{0, 1\}^{50000}$
- If it is to model x explicitly with a multinomial distribution over 2^{50000} possible outcomes, then we would end up with $(2^{50000} - 1)$ dimensional parameter vector!!!

- To model $p(x|y)$, we make a strong assumption
We assume that x_i 's are conditionally independent given y
- This assumption is called **Naive Bayes (NB) assumption**

If $y = 1$ means a spam email; “buy” is word 2087 and “price” is word 39831; then we are assuming that if I tell you $y = 1$, then knowledge of x_{2087} (knowledge of whether “buy” appears in the message) will have no effect on your belief of x_{39831} .

- Formally, $p(x_{2087}|y) = p(x_{2087}|y, x_{39831})$
- $$\begin{aligned} p(x_1, \dots, x_{50000}|y) &= p(x_1|y)p(x_2|y, x_1)p(x_3|y, x_1, x_2) \cdots p(x_{50000}|y, x_1, \dots, x_{49999}) \\ &= p(x_1|y)p(x_2|y)p(x_3|y) \cdots p(x_{50000}|y) \\ &= \prod_{i=1}^n p(x_i|y) \end{aligned}$$

- Our model is parameterized by
 - $\phi_{i|y=1} = p(x_i = 1|y = 1)$
 - $\phi_{i|y=0} = p(x_i = 1|y = 0)$, and
 - $\phi_y = p(y = 1)$
- Given a training set $\{(x^{(i)}, y^{(i)}); i = 1, \dots, m\}$, we write joint likelihood of the data

$$\mathcal{L}(\phi_y, \phi_{i|y=0}, \phi_{i|y=1}) = \prod_{i=1}^m p(x^{(i)}, y^{(i)})$$

- Maximizing $\mathcal{L}(\phi_y, \phi_{i|y=0}, \phi_{i|y=1})$ wrt $\phi_y, \phi_{i|y=0}$, and $\phi_{i|y=1}$ yields

$$\begin{aligned} \phi_{j|y=1} &= \frac{\sum_{i=1}^m 1\{x_j^{(i)} = 1 \wedge y^{(i)} = 1\}}{\sum_{i=1}^m 1\{y^{(i)} = 1\}} \\ \phi_{j|y=0} &= \frac{\sum_{i=1}^m 1\{x_j^{(i)} = 1 \wedge y^{(i)} = 0\}}{\sum_{i=1}^m 1\{y^{(i)} = 0\}} \\ \phi_y &= \frac{\sum_{i=1}^m 1\{y^{(i)} = 1\}}{m} \end{aligned}$$

- $\phi_{j|y=1}$ represents the fraction of spam emails in which word j appears

- Once the value of ϕ_y , $\phi_{i|y=0}$, and $\phi_{i|y=1}$ is known, to make a prediction on a new example with features x , we calculate

$$p(y = 1|x) = \frac{p(x|y = 1)p(y = 1)}{p(x)}$$

$$= \frac{(\prod_{i=1}^n p(x_i|y = 1)) p(y = 1)}{(\prod_{i=1}^n p(x_i|y = 1)) p(y = 1) + (\prod_{i=1}^n p(x_i|y = 0)) p(y = 0)}$$

- Pick whichever class has higher posterior probability
- Naive Bayes algorithm can be generalized to the case where $x_i \in \{1, 2, \dots, k\}$, by modeling $p(x_i|y)$ as multinomial rather than Bernoulli

Living area (sq. meter)	< 400	400-800	800-1200	1200-1600	> 1600
x_i	1	2	3	4	5

- When the original, continuous-valued attributes are not well modeled by MVG distribution, discretizing the features and using Naive Bayes will results in better classifier than GDA

Laplace Smoothing

Consider the scenario wherein you have trained your NBC with a dataset of Spam/Non-spam emails. Now assume that at the end of this course you submitted a paper to NIPS, and you started getting emails with “nips” as a part of your email body. None of your training set has word NIPS. What will be the label of new mail you are getting from NIPS: spam or non-spam?

- Assume that “nips” is the 3000th word in the dictionary, your NB filter will compute its maximum likelihood estimate of the parameters $\phi_{3000|y}$ as

$$\phi_{3000|y=1} = \frac{\sum_{i=1}^m 1\{x_{3000}^{(i)} = 1 \wedge y^{(i)} = 1\}}{\sum_{i=1}^m 1\{y^{(i)} = 1\}} = 0$$

$$\phi_{3000|y=0} = \frac{\sum_{i=1}^m 1\{x_{3000}^{(i)} = 1 \wedge y^{(i)} = 0\}}{\sum_{i=1}^m 1\{y^{(i)} = 0\}} = 0$$

- Why?

- Since your NBC has not seen “nips” before either in spam/non-spam training examples, it thinks the probability of seeing it in either type of email is zero
- To decide whether the new mail containing “nips” is spam, it computes the posterior probabilities as

$$p(y = 1|x) = \frac{(\prod_{i=1}^n p(x_i|y = 1)) p(y = 1)}{(\prod_{i=1}^n p(x_i|y = 1)) p(y = 1) + (\prod_{i=1}^n p(x_i|y = 0)) p(y = 0)}$$

$$= \frac{0}{0}$$

$$p(y = 1|x) = \frac{p(x_1|y = 1) \dots \underbrace{p(x_{3000}|y = 1)}_0 \dots p(x_n|y = 1)p(y = 1)}{p(x_1|y = 1) \dots \underbrace{p(x_{3000}|y = 1)}_0 \dots p(x_n|y = 1)p(y = 1) + p(x_1|y = 0) \dots \underbrace{p(x_{3000}|y = 0)}_0 \dots p(x_n|y = 0)p(y = 0)}$$

Laplace Smoothing

- It is statistically a bad idea to estimate the probability of some event to zero just because you have not seen it in your training set
- Consider the problem of estimating mean of a multinomial random variable z by taking values in $\{1, \dots, k\}$
- We can parameterize our multinomial with $\phi_i = p(z = i)$
- Given a set of m independent observations $\{z^{(1)}, \dots, z^{(m)}\}$, the maximum likelihood estimates are given by

$$\phi_j = \frac{\sum_{i=1}^m 1\{z^{(i)} = j\}}{m}$$

- If we use these MLEs then some of the ϕ_j 's may end up as zero, which is a problem
- To avoid this problem, we can use **Laplace smoothing**, which replaces the above estimate with

$$\phi_j = \frac{\sum_{i=1}^m 1\{z^{(i)} = j\} + 1}{m + k}$$

$$(\sum_{j=1}^k \phi_j = 1 \text{ still holds})$$

Naive Bayes Classifier with Laplace Smoothing

When we apply Laplace smoothing to Naive Bayes classifier, we obtain the following estimates

- $\phi_{j|y=1} = \frac{\sum_{i=1}^m 1\{x_j^{(i)} = 1 \wedge y^{(i)} = 1\} + 1}{2 + \sum_{i=1}^m 1\{y^{(i)} = 1\}}$
- $\phi_{j|y=0} = \frac{\sum_{i=1}^m 1\{x_j^{(i)} = 1 \wedge y^{(i)} = 0\} + 1}{2 + \sum_{i=1}^m 1\{y^{(i)} = 0\}}$
- Does it matter to apply Laplace smoothing to ϕ_y ?

Event Models for Naive Bayes Text Classification

There are two event models for text classification :

- 1 Multi-variate Bernoulli model, and
- 2 Multinomial model

Multivariate Bernoulli Model for Text Classification

- Here we consider a document as an **event**, and the absence or occurrence of words to be attribute of the event
- A document is represented by a vector of binary attributes indicating which words occur and do not occur
- Frequency of a word is not captured
- Here a document can be seen as a collection of **multiple independent Bernoulli experiments**, one for each word in the vocabulary with the probabilities of these word defined by each component, $p(x_i|y = j) \quad j \in \{0, 1\}$
- When calculating the probability of a document, one multiplies the probability of all the attributes values, including the probability of non-occurrence for words that do not occur in the document

Sanjay Singh

Generative Learning Algorithms

- In this model, we assume that the way an email is generated is that first it is randomly determined (acc. to class priors $p(y)$) whether a spammer or non-spammer will send you your next message
- Person sending the email runs through the dictionary, deciding whether to include each word i in that email independently and according to probability $p(x_i = 1|y) = \phi_{i|y}$
- The probability of a message is given by

$$p(y) \prod_{i=1}^n p(x_i|y)$$

Sanjay Singh

Generative Learning Algorithms

- A document is represented by set of word occurrences from the document
- Here, word frequency is considered
- We consider the individual word occurrence to be the **events**, and the document to be the collection of word events
- When calculating the probability of a document, one multiplies the probability of words that occur

- Let x_i is the i -th word in the email
- x_i is an integer taking values in $\{1, \dots, |V|\}$, where $|V|$ is the size of the vocabulary (dictionary)
- An email of n words is represented by $(x_1, x_2, \dots, x_n)$
- In multinomial event model, we assume that the way an email is generated is via a random process in which spam/non-spam is determined before
- Sender of the email writes the email by first generating x_1 from some multinomial distribution over words $p(x_1|y)$
- Next, the word x_2 is chosen independent of x_1 but from the same multinomial distribution, and similarly x_3, x_4 etc untill all n words of the email is generated
- Overall probability of a message is given by $p(y) \prod_{i=1}^n p(x_i|y)$, this formula look similar to the multivariate Bernoulli model but it is different

- Parameters of our new model are
 - $\phi_y = p(y)$
 - $\phi_{k|y=1} = p(x_j = k|y = 1)$ for any j , and
 - $\phi_{k|y=0} = p(x_j = k|y = 0)$
- Assume that $p(x_j|y)$ is same for all values of j , i.e the distribution according to which a word is generated does not depend on its position j within the email

- For a training set $\{(x^{(i)}, y^{(i)}); i = 1, \dots, m\}$
 where $x^{(i)} = (x_1^{(i)}, x_2^{(i)}, \dots, x_{n_i}^{(i)})$, n_i is the number of words in the i -th training example
- Likelihood of the data is given by

$$\begin{aligned}
 \mathcal{L}(\phi, \phi_{k|y=0}, \phi_{k|y=1}) &= \prod_{i=1}^m p(x^{(i)}, y^{(i)}) \\
 &= \prod_{i=1}^m p(x^{(i)}|y^{(i)}; \phi_{k|y=0}, \phi_{k|y=1}) p(y^{(i)}; \phi_y) \\
 &= \prod_{i=1}^m \left(\prod_{j=1}^{n_i} p(x_j^{(i)}|y^{(i)}; \phi_{k|y=0}, \phi_{k|y=1}) \right) p(y^{(i)}; \phi_y)
 \end{aligned}$$

Maximizing $\mathcal{L}(\phi, \phi_{k|y=0}, \phi_{k|y=1})$ yields the maximum likelihood estimates of the parameters:

$$\begin{aligned}\phi_{k|y=0} &= \frac{\sum_{i=1}^m \sum_{j=1}^{n_i} 1\{x_j^{(i)} = k \wedge y^{(i)} = 0\}}{\sum_{i=1}^m 1\{y^{(i)} = 0\}n_i} \\ \phi_{k|y=1} &= \frac{\sum_{i=1}^m \sum_{j=1}^{n_i} 1\{x_j^{(i)} = k \wedge y^{(i)} = 1\}}{\sum_{i=1}^m 1\{y^{(i)} = 1\}n_i} \\ \phi_y &= \frac{\sum_{i=1}^m 1\{y^{(i)} = 1\}}{m}\end{aligned}$$

- When we apply Laplace smoothing (which is needed in practice) to estimate $\phi_{k|y=0}$ and $\phi_{k|y=1}$, we add 1 to the numerators and $|V|$ to the denominator, and obtain
- $$\phi_{k|y=0} = \frac{\sum_{i=1}^m \sum_{j=1}^{n_i} 1\{x_j^{(i)} = k \wedge y^{(i)} = 0\} + 1}{|V| + \sum_{i=1}^m 1\{y^{(i)} = 0\}n_i}$$
- $$\phi_{k|y=1} = \frac{\sum_{i=1}^m \sum_{j=1}^{n_i} 1\{x_j^{(i)} = k \wedge y^{(i)} = 1\} + 1}{|V| + \sum_{i=1}^m 1\{y^{(i)} = 1\}n_i}$$
- NBC is not the very best classifier algorithm, but surprisingly it works well
- It is also a very good “first thing to try,” given its simplicity and ease of implementation